Work Timeline and notes

1. Engineering Trainee - August 2021 to October 2021

- Basic C/C++ concepts
- Linux fundamentals - Basic Linux (how to use), shell scripting, POSIX fundamentals
- ROTS basics and programming - Programming on the STM32 board.

2. Embedded Software Engineer - October 2021 to December 2024

Honeywell, first half overview from October 2021

- Learning about the existing code base, mainly focused on the ONVIF server and other module integration/communication. In which the flow goes like “Web request -> ONVIF Server accept -> Create new thread for request (All operations synchronized properly) -> Parse request XML through GSoap -> Validate inputs -> Pass request to respective module (recorder, streaming, camera settings (to HAL), network, event manager, video analytics, firmware update, user management, etc).
- Documentation of supported ONVIF profiles - G, S, T, M, etc.
- Bug fixing - Identify in which module bug is starting from onvif server to validating inputs to respective modules. Check for different test cases and identify problem. Fix problem and check test cases of that API and APIs around that.
- Self learning of linux fundamentals - Why Linux is used (open source, lightweight, etc), commands in depth, file system, and working with filesystem (access rights, ownership, etc), shell script basics, process control depth (Process memory structure, ps, kill, PID, content under process memory, FDs, etc), user groups, network fundamentals (ping, netstat, ss, ifconfig, ip, DNS, SSH, FTP, SCP), system init flow (boot process), etc.
- Self learning of POSIX thread APIs - Signals and IPC, threads (Multithreading, synchronization, thread management), mutex, conditional variables, semaphores, message queue, shared memory.
- Self learning Socket programming - Network communication using socket, IP, PORT. Learned about TCP and UDP protocols. Server-client architecture Server (socket(), bind(), listen(), accept()), client (socket() and connect()).

  Awards: Awarded power trainee during this period for quickly grasping into real project, which was already in production and further development from a batch of around 70 people.

Honeywell, other half overview till November 2022

- Development of new ONVIF server APIs, completing the support of profile G, S, T, M, etc.

- Development for custom APIs for UI and cross-functional NVRs that connect to the cameras. ONVIF server interfacing with GSOAP and Lighttpd.
- Bug fixing and development of new requirements for different modules of IP camera.
- Development of streaming module that includes the RTSP protocol using live555 and ffmpeg.
- Debugged a server issue - The TCP connections were stuck in CLOSE_WAIT because a specific API thread was hanging and failed to call close() on the socket after receiving the client's FIN, leading to resource leakage and preventing new client connections. (Learned about TCP flow, sockets, etc)
- Merged the codebase of 3 series cameras into one, increasing the manageability of code. Made plans to merge database usage, settings, etc (Made everything dynamic).
- Added dynamic support of multisensor cameras, refactored the whole code for multisensor (i.e., database for settings will have settings for each sensor index, get call based on sensor index, API level support based on sensor index, etc) - **Got appreciated for this work as I completed major portiton in very less timeline**.
- Collaborated on firmware update module, OTA update, timed checks, settings, etc.

Duclo, first half overview from November 2022

- Project code overview
- Debugged a critical WebRTC live-stream issue where frames were corrupted and video was unstable before an investor demo. Found that the custom RTP packetizer ignored RTP/FU-A header length during fragmentation, causing payload misalignment and incorrect stop-bit marking. Fixed the calculation, ensuring proper frame boundaries and smooth, stable streaming. The client was very happy, as it was a long-standing issue, and gave appreciation though that I got a **pat on the back award**.
- Design and development of multi-client live streaming, where the camera can handle multiple WebRTC live stream requests, made a list for client handling in C language. And made the request handler multithreaded. Challenge - the old structure was only able to serve one request at a time, the variables were global and fixed, created structures and treated each request as a different object, and modified the whole structure as per this.
- R&D for the CVR module, where I tried a reverse-SSH tunnel to send a video stream from live555 to the CVR server. The ssh tunnel was unstable and connecttion was breaking due to several issues, also we cannot change the network configs in an organization.

Duclo, other half overview till December 2024

- Finally, we engineered a Cloud Video Recorder module with an RTSPS push mechanism using C++ classes, queues, and threads for buffer management and event-based recording, while developing TLS and digest authentication for secure communication and ensuring network failure resilience with a minimum 30-second buffer. Here, I collaborated with the cloud team, which was responsible for the CVR server, to decide on protocol, security enhancement, and video timestamp (done

through extension header right now). During this, I learned the RTSP protocol in depth.
- Designed an event manager module that takes events from rtsp channel, formats it to the specified JSON, and sends them to the database. For this I designed a custom lightweight XML to JSON parser to convert existing XML data to JSON.
- Made an entitlement manager module, offering access control over 5+ different functionalities like CVR, live streaming, events, video analytics, etc.
- Designed an event subscription model, in which a client can subscribe to a specific type of event for their devices, like a smartphone app/web. Designed subscriber management, event filtering, event message format, and keepalive methodology to reduce load.
- Implemented an MQTT-based settings module for cloud app integration, enabling centralized configuration management across over 4+ SKUs for Hanwha.
- Collaborated on the cloud snap upload module, where we used a thread pool to upload snaps to the cloud at a specified period.

Academic Timeline and Notes

B.Tech., Electronics & Communication Engineering — Nirma University (July 2017 – June 2021)

Year 1–2 (Foundation Years):

Built fundamental understanding of electronics and computing. Explored **Digital Logic Design**, **C programming**, and **Basic Electrical Circuits**, emphasizing structured programming and algorithmic thinking.

Developed curiosity in **embedded systems** through introductory **8085/8086 assembly**, **microcontroller labs**, and **sensor interfacing experiments**.

Year 3 (Core Embedded & Networking Concepts):

Learned **Data Communication & Networking**—protocol layering, TCP/IP stack, socket APIs—and implemented small simulations in C.

Completed **Embedded Systems Lab** using **AVR and ARM Cortex-M microcontrollers**, focusing on **GPIO, timers, UART, I²C, and SPI** programming.

Created early prototypes such as **Person Counter using IR sensors** and **16-bit RISC processor in Verilog**, gaining hands-on practice with Quartus and ModelSim.

Explored **Wireless Sensor Networks**, developing packet-based communication among nodes.

Year 4 (Capstone and Applied Projects):

Led the **Autonomous Underwater Vehicle (AUV)** project as part of the university robotics initiative. Integrated **ROS**, **Jetson TX2**, and **OpenCV** for underwater object detection and control.

Designed **vision and control subsystems**, achieved real-time performance using **CUDA** acceleration.

Submitted and published a paper titled *“Industrial Automation Using PLC and IoT Advancements”* in IJERT (2021).

Graduated with a final project grade A and a practical focus on system integration, signal processing, and embedded software.

M.S., Computer Science & Engineering — University at Buffalo (2025 – Present)
Core Systems and Security Focus:
Deep dive into **Operating Systems**, **Computer Architecture**, and **Modern Networking Concepts**, reinforcing low-level and concurrency principles through **Pintos kernel projects** and **NS-3 simulations**.
Worked on kernel-level synchronization primitives, CPU scheduling algorithms (priority donation, load average), and user program system call handling.
In **Computer Security**, analyzed real-world attacks (HBGary hack, ATM security model) and practiced cryptography (AES, RSA, CBC/ECB modes) and public-key infrastructure.

Data-Intensive and Applied ML Coursework:
Handled large-scale datasets in **Data Intensive Computing**, building pipelines with **pandas, scikit-learn**, and **Spark-style frameworks**.
Developed the **Digital Well-being Data Analysis Project (300K+ records)** investigating correlations between screen time and sleep quality using **XGBoost**, **SVM**, and clustering algorithms.

Entrepreneurship and Innovation Track:
Completed the **Dream-Plan-Do** framework under UB's **ENT program**, applying innovation tools like **Jobs-To-Be-Done (JTBD)**, **Hunting Zones**, and **Backcasting**.
Built business-technology bridges by analyzing market data (e.g., IoT dog toy case study) and modeling performance ceilings for product growth.

Research and Teaching Interests:
Exploring how **embedded systems and cloud streaming architectures** intersect with **edge AI** and **secure media transport**, aiming for future academic or industry research in real-time computer vision pipelines and resilient distributed systems.